

T12 — Ereditarietà, Dynamic Binding e Polimorfismo

Estensione di Classi, super, Overriding vs Overloading, Polimorfismo e MavenDate v3

Docente: Prof. Michele Pasqua — **Appunti Revisione:** Wally

A.A. 2025/2026 – Università degli Studi di Verona

In questo blocco analizziamo in modo esaustivo i due pilastri più importanti dell'OOP in Java: l'**Ereditarietà** e il **Polimorfismo** ([T12 - Inheritance and Polymorphism.pdf](#)). Nel codice di [Lezione12/MavenDate](#), la gerarchia evolve introducendo la classe astratta intermedia `FormattedDate`, l'interfaccia `Time`, la classe estesa `TimeStamp` e l'interfaccia standard `Comparable` per l'ordinamento naturale degli array.

0.0.1 1. Teoria: Concetti Teorici dalle Slide (Slide T12)

A. Ereditarietà di Classe (`extends`) e Riuso (Slide 1–17)

- **Relazione “IS-A” vs “HAS-A”:**

- *Ereditarietà (“IS-A”)*: un'auto è un veicolo (`Car extends Vehicle`); una data italiana è una data (`ItalianDate extends Date`).
- *Composizione (“HAS-A”)*: un'auto ha un motore (`Car` ha un campo `Engine`); una persona ha una data di nascita (`Person` ha un campo `Date`).

- **Ereditarietà Singola in Java:**

- In Java **non esiste l'ereditarietà multipla tra classi** (a differenza del C++). Una classe può estendere al massimo **una sola classe genitore diretta** (`extends SuperClass`). Questo previene il cosiddetto *Diamond Problem*.

- **Overriding vs Overloading:**

- *Method Overloading*: metodi nella stessa classe con lo stesso nome ma parametri diversi (risolto staticamente a compile-time).
- *Method Overriding*: una sottoclasse riscrive l'implementazione di un metodo ereditato mantenendo **la stessa identica segnatura** (nome, tipi dei parametri, ordine) e un tipo di ritorno identico o covariante.
- **L'Annotazione `@Override`**: Non è sintatticamente obbligatoria per far funzionare l'overriding, ma è una best practice fondamentale: dice al compilatore di verificare che quel metodo esista davvero nella superclasse, intercettando errori di battitura (es. scrivere per sbaglio `toString()` con la 's' minuscola verrebbe segnalato come errore).

- **Costruttori ed Ereditarietà: La Parola Chiave `super`:**

- Attenzione: **I costruttori NON vengono mai ereditati dalle sottoclassi.**

- All'interno del costruttore di una sottoclasse, la prima istruzione deve essere una chiamata a `super(...)` oppure a un altro costruttore con `this(...)`.
- Se il programmatore non scrive nulla, il compilatore inserisce automaticamente `super();` (invocazione del costruttore vuoto della superclasse).
- Attenzione: **Trappola d'esame**: Se la superclasse non possiede un costruttore senza parametri (perché ne ha definito uno con parametri e non ha aggiunto quello di default), omettere `super(...)` genera un **Compile-Time Error**!
- `super.metodo()` permette inoltre di invocare l'implementazione originaria della superclasse, bypassando l'override locale.

- **Classi e Metodi final:**

- `final class`: non può essere estesa da nessun'altra classe (es. `String`, `Integer`).
- `final method`: non può essere sovrascritto (*overridden*) da nessuna sottoclasse.

B. La Classe Radice Universale: `java.lang.Object` (Slide 18–31)

- «*The One Ring*»: in Java qualsiasi classe estende implicitamente `java.lang.Object`. Se una classe non specifica `extends`, estende direttamente `Object`.
- **I Metodi Universali di `Object`**:
 - `public String toString()`: rappresentazione testuale (`NomeClasse@hashCode`).
 - `public boolean equals(Object obj)`: confronto semantico (di default confronta l'identità con `==`).
 - `public int hashCode()`: valore hash associato all'oggetto.
 - `public final Class<?> getClass()`: restituisce il descrittore a runtime del tipo della classe.
- **Il Contratto Formale di `equals`**: Ogni implementazione di `equals` deve rispettare 5 proprietà matematiche:
 1. *Riflessiva*: `x.equals(x)` è sempre `true`.
 2. *Simmetrica*: `x.equals(y)` ritorna `true` se e solo se `y.equals(x)` ritorna `true`.
 3. *Transitiva*: se `x.equals(y)` è `true` e `y.equals(z)` è `true`, allora `x.equals(z)` è `true`.
 4. *Consistente*: invocazioni multiple producono lo stesso risultato finché lo stato non muta.
 5. *Non-Nullità*: per qualsiasi `x != null`, la chiamata `x.equals(null)` **deve restituire false** senza lanciare eccezioni.

C. Polimorfismo e Dynamic Method Dispatch (Late Binding) (Slide 32–50)

- **Principio di Sostituzione di Liskov (LSP)**: Se S è sottotipo di T , qualsiasi porzione di codice scritta per interagire con T deve poter operare in modo trasparente e corretto con un'istanza di S .
- **Tipo Statico vs Tipo Dinamico**:

```
1 Date d = new ItalianDate(1, 1, 1970);
```

- **Tipo Statico (a Compile-Time)**: è il tipo dichiarato della variabile reference (`Date`). Il compilatore accetta solo invocazioni a metodi e accessi a campi definiti nella classe `Date`.
- **Tipo Dinamico (a Runtime)**: è il tipo effettivo dell'oggetto allocato nello Heap (`ItalianDate`).
- **Dynamic Method Dispatch (Late Binding)**: Quando a runtime viene eseguita una chiamata a un metodo d'istanza (es. `d.toString()`), la JVM consulta la **Virtual Method Table (vtable)** dell'oggetto concreto nello Heap e seleziona **la versione del metodo definita dal Tipo Dinamico**! Non importa che `d` sia dichiarata come `Date`: verrà eseguito il `toString()` specializzato di `ItalianDate`.

- **Regole di Conversione di Tipo (Casting tra Oggetti):**

- **Upcasting (verso la superclasse):** `Date d = new ItalianDate(...);` $\Rightarrow$ **Implicito, automatico e sempre sicuro al 100%.**
- **Downcasting (verso la sottoclasse):** `ItalianDate id = (ItalianDate)d;` $\Rightarrow$ **Esplicito obbligatorio.**
 - * Se l'oggetto concreto nello Heap è effettivamente un'istanza compatibile, il cast ha successo.
 - * Se l'oggetto concreto nello Heap è di tipo diverso (es. un `Date` generico o un `AmericanDate`), la JVM interrompe l'esecuzione e lancia una **`ClassCastException`**!
- **Downcasting Sicuro con `instanceof`:** Per evitare crash a runtime si verifica preventivamente la compatibilità:

```

1  if (d instanceof ItalianDate) {
2      ItalianDate id = (ItalianDate) d;
3  }

```

0.0.2 2. Codice: Evoluzione del Codice: [Lezione12/MavenDate](#)

In [Lezione12](#) la gerarchia del progetto diventa ricca e completa:

```

1  classDiagram
2      class Object {
3          +toString()
4          +equals(Object)
5      }
6      class Comparable {
7          <<interface>>
8          +compareTo(Object)
9      }
10     class Time {
11         <<interface>>
12         +getSeconds()
13         +getMinutes()
14         +getHours()
15     }
16     class Date {
17         #day: int
18         -month: int
19         -year: int
20         +toString()
21         +equals(Object)
22         +compareTo(Object)
23     }
24     class FormattedDate {
25         <<abstract>>
26         #format: String
27         #months: String[]
28         +printFormat() final
29         +getMonthAsString() final
30         +prettyPrint()* String
31     }
32     class ItalianDate {
33         +prettyPrint()
34         +toString()
35     }
36     class AmericanDate {
37         +prettyPrint()
38         +toString()

```

```

39     }
40     class TimeStamp {
41         #seconds: int
42         #minutes: int
43         #hours: int
44         +toString()
45         +equals(Object)
46     }
47
48     Object <|-- Date
49     Comparable <|.. Date
50     Date <|-- FormattedDate
51     FormattedDate <|-- ItalianDate
52     FormattedDate <|-- AmericanDate
53     Date <|-- TimeStamp
54     Time <|.. TimeStamp

```

1. L'Interfaccia Comparable in `Date.java` `Date` implementa l'interfaccia standard `java.lang.Comparable` per consentire l'ordinamento naturale:

```

1  public class Date implements Comparable {
2      ...
3      @Override
4      public int compareTo(Object other) {
5          Date otherAsDate = (Date) other;
6          int diff = this.year - otherAsDate.getYear();
7          if (diff != 0) return diff;
8          diff = this.month - otherAsDate.getMonth();
9          if (diff != 0) return diff;
10         return this.day - otherAsDate.getDay();
11     }
12 }

```

Logica di confronto: confronta prima gli anni, se uguali confronta i mesi, se uguali confronta i giorni. Restituisce un intero negativo se `this < other`, zero se uguali, positivo se `this > other`.

2. L'Astrazione con Classe Astratta: `FormattedDate.java` Centralizza i dati di formattazione evitando duplicazioni tra `ItalianDate` e `AmericanDate`:

```

1  public abstract class FormattedDate extends Date {
2      protected final String format;
3      protected final String[] months;
4
5      public FormattedDate(int day, int month, int year, String format, String[] months) {
6          super(day, month, year);
7          this.format = format;
8          this.months = months;
9      }
10
11     // Metodi final: le sottoclassi non possono sovrascriverli (Template Method Pattern)
12     public final String printFormat() { return format; }
13     public final String getMonthAsString() { return months[getMonth() - 1]; }
14
15     // Metodo astratto: DEVE essere implementato dalle sottoclassi concrete
16     public abstract String prettyPrint();
17 }

```

3. Ereditarietà Multipla di Tipo: `TimeStamp.java` Mostra come combinare l'estensione di una classe concreta con l'implementazione di un'interfaccia:

```

1 public class TimeStamp extends Date implements Time {
2     protected final int seconds;
3     protected final int minutes;
4     protected final int hours;
5
6     public TimeStamp(int seconds, int minutes, int hours, int day, int month, int year) {
7         super(day, month, year);
8         this.seconds = seconds;
9         this.minutes = minutes;
10        this.hours = hours;
11    }
12
13    @Override
14    public String toString() {
15        // Riutilizza il toString() della superclasse aggiungendo l'orario con zero-padding a due cifre:
16        return String.format("%s[%02d:%02d:%02d]", super.toString(), hours, minutes, seconds);
17    }
18
19    @Override
20    public boolean equals(Object other) {
21        // 1. Verifica prima l'uguaglianza della data con super.equals():
22        if (super.equals(other) == false) return false;
23        if (!(other instanceof TimeStamp)) return false;
24        TimeStamp otherAsTimeStamp = (TimeStamp) other;
25        // 2. Poi verifica l'orario:
26        return hours == otherAsTimeStamp.getHours() &&
27               minutes == otherAsTimeStamp.getMinutes() &&
28               seconds == otherAsTimeStamp.getSeconds();
29    }
30 }

```

4. Dimostrazione Pratica in `MainDate.java`

```

1 Time ts1 = new TimeStamp(0, 0, 10, 10, 11, 2025);
2 System.out.println(ts1.toString()); // Esegue TimeStamp.toString() tramite Late Binding!
3
4 // ts1.getDay(); // COMPILER-TIME ERROR: il tipo statico Time non possiede il metodo getDay()!
5 System.out.println(((Date) ts1).getDay()); // OK: cast esplicito a Date!
6
7 Date[] arr = { ts2, itDate, new Date(9, 9, 2025) };
8 Arrays.sort(arr); // Ordina l'array eterogeneo grazie a compareTo()!
9 System.out.println(Arrays.toString(arr));

```

Output a video prodotto dall'ordinamento polimorfico:

```

1 [y2025m9d9, 3/11/2025, y2025m11d10[10:00:00]]

```

- L'ordinamento colloca correttamente prima il 9 settembre, poi il 3 novembre (itDate) e infine il 10 novembre (TimeStamp).
- In fase di stampa, ciascun elemento viene formattato dal **proprio** metodo `toString()` dinamico.

Nota di Studio

Con la **Lezione 12** abbiamo analizzato la gerarchia completa di ereditarietà, l'overriding con `super`, il late binding, la classe astratta `FormattedDate` e l'ordinamento con `Comparable`.

Il prossimo blocco è la **Lezione 13 / Slide T13: *Abstract Classes and Interfaces***: - Differenze ontologiche e strutturali tra **Classi Astratte** e **Interfacce**. - Metodi `default` e metodi `static` nelle interfacce (Java 8+). - Ereditarietà multipla tramite interfacce e risoluzione dei conflitti (*Diamond Problem* sui metodi `default`). - Evoluzione di `MavenDate` in `Lezione13`: - La classe `Date` viene ripensata per gestire il calendario completo. - Analisi delle nuove interfacce e classi di supporto.

Dammi conferma per aprire la Lezione 13 / T13 e analizzare a fondo Classi Astratte e Interfacce!